

Chapter 7

Better Hedging with Reinforcement Learning

Introduction

A New AI Trading Assistant

When we began writing this book in 2023, it marked the 50th anniversary of the publication of the seminal work by Black and Scholes (1973) and Merton (1973) on the pricing of options. The Black-Scholes Merton (BSM) pricing model ushered in a new era in quantitative finance and capital markets. The impact of their treatise was immediately felt both in academic research and on the market prices of options. It was indeed in the 1970s and 1980s that traders in the options pit acquired a new weapon—the Black-Scholes (*abbr.* BS, which will prove to be a versatile abbreviation) option pricing model, later programmed into a desktop computer, or the Rolodex of personal computing devices, the HP financial calculators. The earliest documented Black-Scholes program for HP calculator was written by Peter Carr, then a doctoral student at UCLA on HP-12C, published in 1988 (Carr, 1988). Type in a few inputs—stock price, expiration date, rate on treasury bills, and a guesstimate volatility—the trusty HP would spit out a price instantaneously. A new “AI” trading assistant had just arrived! To the traders’ surprise, or to some much delight, the prices of options they were trading with each other were...

wrong!?

The fact that the market prices were much different from BS prices was an opportunity for windfall profits, but only if the traders knew how to hedge their positions and wait for prices to converge to the “arbitrage-free” BS prices. This was a problem because the BSM model assumes continuous hedging in a frictionless (i.e., zero transaction cost) market with fractional shares. This market did not exist at the time, has not come to being since, and we can state with a high degree of certainty, will not exist in the near future—even with zero-commission, miniscule bid-ask spread, fractional shares, and high-frequency trading technologies, we are still qualitatively away from the ideal. But no matter, the pit traders who normally would look down on, or rather as one former pit trader

would quip, “look up and thumb their noses at” the academics and dismiss their fantastical models became infatuated with the BS (as we said earlier, this abbreviation is versatile). As a result, the BS price became a self-fulfilling prophesy. The rest is history. BSM now permeates unexpected corners of financial economics, accounting, and corporate finance, from pricing employee stock options, evaluating merger and acquisition deals, to justifying the stratospheric valuations of unicorn startup ventures.

Continuous Hedging Is Not Required

To be fair, BSM does not necessarily require that one carries out continuous and frictionless hedging. In fact, you do not need to hedge at all. If a market is reasonably efficient, everyone agrees on the probability of distribution of outcomes at expiration of the option contracts, and the “implied” volatility during the contract’s lifetime is constant, then market prices should converge to BS prices. Note, this convergence has a random distribution and is not arbitrage-free. That is because, in finance, we discount future cash flows based on uncertainty and risk of such expected cash flows. BSM is consistent with the discount cash flow analysis. And the one-price theory predicts that in a perfectly competitive market, traders will converge on the same set of “equilibrium” expectations about prices, volatilities, and risk premia. Still there are many assumptions here that are fundamentally the same as those of BSM.

Already, practitioners noticed inconsistencies in the assumptions of BSM. For example, when using BSM to calculate the “implied volatility (vol)” of a particular stock from the market price of options, one notices that implied vol are different at different strike prices. Indeed, there appears a “volatility smile” where implied vol seems to increase further in (or out) of the money ([Figure 7.1](#)). The volatility smile is inconsistent with the “constant volatility” assumption. There is also a volatility skew, normally negative, that is, the implied vol for an out-of-the-money put is higher than implied vol of an out-of-the-money call with the same “money-ness” (i.e., the difference between strike and underlying prices), and equivalently, the implied vol of an out-of-the-money call is higher than the implied volatility of in-the-money call with the same absolute moneyness ([Figure 7.1](#)). This skew is not explained by the skew in the return distribution of the underlying prices, and hints at risk premium skew or more fashionably, a behavioral bias, which can only persist if they cannot be arbitrated away.

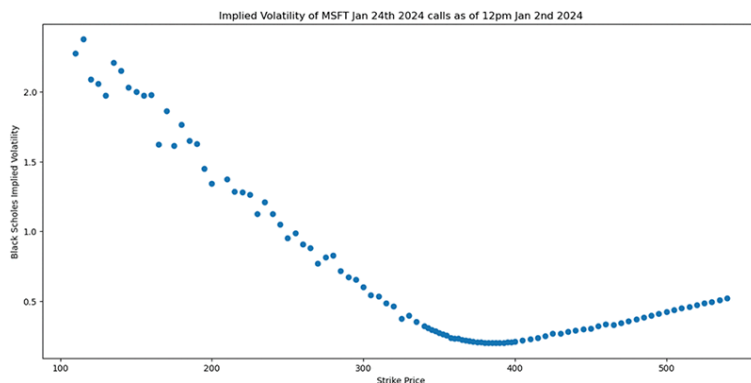


Figure 7.1 The volatility “smile” and “skew”: implied volatility of MSFT call options expiring on January 24, 2024, as of 12:00 pm on January 2, 2024. MSFT stock price was about \$375/share. Implied volatility calculated using BSM from the mid-prices of call options.

Continuous hedging makes the assumptions of asset return distribution unnecessary in theory. (You still must deal with the distribution of asset price paths, which relies on a similar set of assumptions.) Advocates of Black-Scholes and Merton’s approach, therefore, would tweak the underlying assumptions; for example, Heston (1993) proposed a stochastic process for the volatility of the underlying asset and a numerical algorithm to calibrate model parameters to observed market prices of options. Empirically, the Heston model fits observed market data quite well. But the improvement comes at the cost of increased model complexity, which is harder and more costly to work with, as numerical calibration tends to consume a lot of computational resources. And there is a big problem with these calibrated models: they tend to be more complex and have more parameters. For Heston, the parameter values for different stocks turned out to be different, hence the risk of overfitting!

Then, there are costs associated with hedging trades, including commissions, exchange fees, bid-ask spreads, impact of trades on market prices, and over- and under-hedging due to share count rounding. To properly account for these costs, one needs to simulate potential trading costs realistically. Since different traders face different trading costs, a robust estimation of the distribution of trading costs is also needed.

Deeper and deeper into the rabbit hole goes Alice!

Machine Learning Comes to the Rescue

With the onset of machine learning (ML), particularly with reinforcement learning (RL), it is now feasible to predict option prices without the structural models like BSM or Heston. We can now incorporate fundamental and empirical features previously ignored in these models, such as transaction costs, liquidity, market sentiment, and so on, into a deep neural network (DNN). We let the DNN learn from market data, and then price arbitrary options through interpolation and extrapolation.

Not only can an artificial intelligence (AI) “agent” learn a realistic pricing model (although we may not know exactly how it does so), we can also use reinforcement learning to train the AI agent to construct an optimal hedging policy *ex ante*, incorporating aforementioned practical considerations of transaction costs and market friction. The result is a much more

robust hedging policy that market makers and options dealers can deploy programmatically to hedge the risk of taking on inventory, thereby increasing the overall liquidity and efficiency of the options market.

Furthermore, the method we describe here can be generalized to combinations of options (i.e., combos) or structured notes. Structured notes are complex securities, usually mature in 3–5 years, and make coupon and principal payments that are indexed to individual stock prices or market indices, and such coupon and principal payments are usually contingent upon the future prices of reference stocks or indices reaching certain threshold. The contingent payment structure are embedded options. For example, a structured note issued by Barclays referenced to the S&P 500 Index has the following conditional principal value matrix:

$$V = \begin{cases} V_0 + (x - V_0) * 1.105 & \text{if } x > V_0 \\ \min(0, x - 0.8 \times V_0) & \text{if } x \leq V_0 \end{cases},$$

where V_0 and x are the initial and final value of the reference indices, respectively. This note provides leverage (1.105) of the reference index return and a protective buffer of 20%, which is effectively a long and short pair of put options: a long an at-the-money put, and a short an out-of-the-money put, with a strike price at 80% of initial value of the reference index, V_0 . Like options market makers, issuers of such notes have a need to hedge.

In this chapter, we will describe a “simplified” reinforcement learning algorithm, implemented and made available on [QuantConnect.com](https://www.quantconnect.com) for practitioners of options trading. We will discuss the underlying algorithmic framework, present example codes, and investigate the performance of the reinforcement learning hedging algorithm, compared to more traditional hedging based on matching delta, or, delta hedging. Before proceeding with the rest of the chapter, we encourage the readers to familiarize themselves with the basic option pricing theory like the BSM and machine-learning concepts of neural network (NN) and reinforcement learning (RL).

A Simplified but Effective Reinforcement Learning Approach

First things first, due to the complexity and cost of large RL models required to train an industrial-strength AI hedging algorithm, a comprehensive treatment of state-of-the-art AI hedging is beyond the scope of this book, and beyond the capability and resources of lone-wolf quant traders. Instead, the author and his research team have developed an approach that simplifies RL, with a reduced NN architecture, and reliant on a much smaller dataset, that can be implemented effectively by the reader.

The models and algorithms presented here have been incorporated into the online algorithmic trading library of the [QuantConnect.com](https://www.quantconnect.com) platform, which seamlessly integrates with the platform’s extensive market data for options and equities. In what follows, we shall use the source code implemented on [QuantConnect.com](https://www.quantconnect.com) as examples. However, if readers so choose, they can implement the code in a Python environment on a local computer with a GPU and a minimum of 16 GB of RAM. Readers will also need access to daily options and equities data for individual tickers of

their choice. The entire Python files associated with this example are available in the book's GitHub repository at [gnt.co/book-repo](https://github.com/gnt-co/book-repo). For instructions on how to clone the repository, please refer to “Part III Part Overview.”

Overview of the Reinforcement Learning

In general, a typical RL AI hedging program contains these steps:

1. Identification
2. Training with simulated data
3. Training or refinement with “real” market data
4. Testing and implementation

We will describe these steps conceptually in this section and illustrate them with code examples in the following section.

Identification

This step identifies and specifies the price processes of underlying stocks. For brevity and readability, we shall use the term “stock” to describe all reference assets or indices of options. The underlying stocks are risky assets, and their price processes are stochastic in nature. Since we shall admit potentially different asset classes as the underlying, price dynamics can differ significantly. Therefore, it is important in this step to identify the specific process(es) and estimate or calibrate parameters that are sufficiently accurate in describing the process and distribution of outcomes.

Without loss of generality, in the following example, we adopt the Geometric Brownian Motion (GBM) as the underlying process for stocks:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

where S_t is the stock price at time t , μ is the mean drift of the stock return, σ is the volatility of the stock return, and W_t is a Wiener process (Brownian motion). The return of the stock is distributed with a mean of μ and standard deviation of σ .

Other stochastic processes of underlying stocks can be adopted as well. For example, the Heston model (Heston, 1993) posits:

$$dS_t = \mu S_t dt + \sqrt{v_t} S_t dW_t^S.$$

The Heston model relaxes the assumption that the volatility of stock return is invariant. Instead, the model added an additional time-dependent stochastic process for volatility, $\sigma_t = \sqrt{v_t}$.

$$d\sqrt{v_t} = -\theta\sqrt{v_t}dt + \delta dW_t^v$$

The volatility is modeled as an Ornstein-Uhlenbeck process, with parameters θ and δ , the variance of the Wiener process for volatility. The negative sign in front of θ suggests that this is an auto regressive process. Indeed, Ornstein-Uhlenbeck is the continuous-time version of discrete autoregressive process of order 1, AR(1). The two Wiener processes for stock, W_t^S and volatility W_t^v are independent. This double-stochastic

process provides a more market consistent way to model stock prices but at a (much) higher cost of computation, for calibration of parameters from market data, and for numerical estimation of options prices and other analytics (Greeks). Therefore, in this simplified model, we have chosen the more analytically appealing GBM, which requires only traditional statistics to estimate parameters and has closed-form solutions for option prices and Greeks, that is, the Black-Scholes Merton model.

Simulation

It is a good idea to simulate training data, when the hedged instrument (in this example, a call option) is a derivative of an underlying security (in this example, a stock) where a theoretical no arbitrage relationship can be specified. Despite the appearance of large data, historical financial market data is actually quite inadequate for reinforcement learning. This is because financial markets are competitive. Any “innovation,” i.e., deviations from fundamental price relationships, that may reasonably persist (e.g., in a time frame that an algorithm can react to and trade on) are usually very hard to detect. Or put it another way, the signal-to-noise ratio of such a tradeable innovation is very low. Therefore, most financial datasets contain only low information content for models to reliably learn persistent innovation that are not already known to structural models. If our goal is to train a hedging policy to respond to some known, persistent non-structural behavior, then we can simulate these “known unknowns,” as long as we correctly identify them.

In the example here, the persistent and fundamental behavior we want the model to learn is dominated by the no-arbitrage relationship between the prices of options and underlying stocks: no arbitrage relationship can be approximated using different models (we discussed BSM and Heston in the last section). We identified one, BSM with GBM, as the core relationship between options and stock prices. Hence, we will simulate copious amounts of data using BSM with GMB, and first train the model to find the best hedging strategy under these assumptions. We had earlier discussed the shortcomings of theoretical models and clarified the motivation of using RL to learn more realistic aspects of hedging. So, what do we gain from training a model on simulated data?

The answer is nuanced. The first part of the answer is that one aspect of “realism” is discrete hedging, as opposed to continuous hedging that underlies BSM. With simulated data, we simulate discrete steps in asset prices (daily in this example) and discrete hedging (daily also in this example). It is possible to determine analytically the “optimal” discrete hedging strategy. But the computation is quite complex, requiring solving stochastic partial differential equations that, in most cases, can only be performed numerically. This makes it impractical to implement, as the latency between an update to input states and the decision of appropriate hedge action is unacceptably long if the trading frequency is minutes or seconds. An ML policy network, on the other hand, once learned, can respond to current market conditions (states) and derive “actions” with very low latency.

The other answer is, because we are simulating data, any known variation or innovation to the fundamental pricing model, such as liquidity, market trend or other predictive models, transaction cost model, and so on, can be built into the simulator. The caveat is they must be “known unknowns.” RL models will not be able to learn from simulated data, those processes that have not been seen. This brings us to the next topic: refinement training on actual market data.

Refinement Training on Actual Market Data

In situations where we believe large datasets of historical market prices contain persistent innovation or processes unknown to us, we can train the model on “real” market data. As we discussed previously, the competitive nature of the financial markets makes it unlikely for any known or unknown unknowns to persist and to be noticeable in data. In certain situations, where there are a lot of data and the market is not perfectly competitive, for example, in high-frequency (HF) trading, where the investments in high-end computing and information technology hardware required make the HF market place less competitive, ML/RL models trained on real data, tick-by-tick or high-frequency time-bar data, can yield significant advantages over competitors’ rule-based algorithms. But even in this case, simulation can play an important role in priming the model with a theoretical and fundamental prior. Then, the model can be refined on precious real market data to incorporate real-world challenges and additional “unknown unknowns.”

In this example, we incorporate market data in the second “refinement” stage of training a hedging policy network. In this refinement stage, a short window of historical market data is used to fine-tune the policy network to recent information. We chose a short and recent historical window because option markets have been evolving rapidly, and the features, such as liquidity, transaction costs, and market trends, which we want the RL agent to learn do not stay stationary. We will discuss details of the implementation in the next section.

Testing and Implementation

As with all machine-learning efforts, it is best practice to split sample data into training, validation, and out-of-sample testing. In the case of financial market applications, since the market is continuously generating new data, we have the opportunity to conduct out-sample tests using new market data. This is the age-old tradition of running a “paper” portfolio to continuously monitor the out-sample performance of algorithmic trading models (or fundamental investment processes) and use results of the out-sample to (in)validate, refine, and enhance the model.

Even upon implementation of a trading algorithm with real money, we should continue to systematically analyze performance results. It is important to understand the variance between what is expected of a model performance conditioned on the current market state (*ex ante*), and what we experience in real money trading (*ex post*), to help identify blind spots, inaccuracies, or even errors in our models. We will not cover this

part of the AI hedging process, as the details depend on application. We would like to emphasize that no AI hedging should be implemented without a well-thought-out plan for continuous testing, attribution, and model refinement in place.

Now we proceed to illustrate AI hedging with the code example implemented on [QuantConnect.com](https://www.quantconnect.com).

Implementation on QuantConnect

This example is implemented on [QuantConnect.com](https://www.quantconnect.com) and uses PyTorch's NN interface for RL. We will not cover the basics of PyTorch in this book. The readers are directed to the excellent online documentation and tutorial at pytorch.org.

The objective of this project is to obtain an optimal hedging policy that minimizes the variance of the profit and loss (PnL) of a hedged portfolio containing the asset to be hedged, in this case a call option, and the hedging instrument, in this case, the underlying stock. The example demonstrates the process of training an agent, or hedging policy network with AAPL (Apple, Inc.) call options and stock. In the following text, we use the terms “agent” and “policy network” interchangeably. But there is a slight distinction: the policy network refers to the NN whose parameters are tuned through RL, where the input is the current market state, and the output is the hedging decision for the next period. The “agent” refers to the trading algorithm armed with the policy network to make real-time decisions and trades on those decisions. In other words, the “agent” is the AI.

The reader will find the entire project on [QuantConnect.com](https://www.quantconnect.com)'s code library under “aihedging.” The organization of the files is as follows:

AI Delta Hedging. This is the main project:

- **research.ipynb:** the main model to run the two-stage training, test the model, and generate the results.
- **main.py:** defines the class, `AIDeltaHedgingAlgorithm`, which contains the initialization of training and sets up general parameters. The class can be called in the backtest environment on [QuantConnect.com](https://www.quantconnect.com).
- **aihedging:** a separate project containing the class and function definitions of components of the main research notebook.
 - **model.py:** defines the classes and functions called by the main research notebook.
 - **policy.py:** the “policy” class that defines the architecture of the policy neural network.

We will describe each component next.

Primary Research Notebook

In the research notebook, `research.ipynb`, we will illustrate the process of the two-stage training, and some simple tests of the model performance. The first cell in the research notebook creates the model, fixes the ran-

dom seed to ensure reproducibility, and chooses the processor, e.g., CPU or GPU. We will look at the details of the `AIDeltaHedgeModel` class later.

[research.ipynb/Cell 1]

```
from aihedging.model import AIDeltaHedgeModel

qb = QuantBook()

model = AIDeltaHedgeModel(
    qb, timedelta(qb.get_parameter('min_contract_duration', 30)),
    timedelta(qb.get_parameter('max_contract_duration', 120)),
    timedelta(qb.get_parameter('min_holding_period', 14))
```

The second cell trains the base model, or the first stage of training the policy network to mimic delta hedging. We will look at the details of the `model.train_base_model` function later.

[research.ipynb/Cell 2]

```
model.train_base_model(plot=True, epochs=1000)
```

The third cell runs the refinement stage, where we apply a penalty function based on PnL of the hedged portfolio, and refine the policy network with daily closing prices of AAPL (Apple Inc.) from December 19, 2023, to February, 17, 2024. We will look at the details of the `model.train_asset_model()` function later.

[research.ipynb/Cell 3]

```
start_date = datetime(2023, 12, 19)
end_date = datetime(2024, 2, 17)

model.train_asset_model("AAPL", start_date, end_date,
    epochs=40, save=False, in_research_env=True)
```

The last cell runs tests of the performance of the model. We will look at the details of the `model.research_test()` function later.

[research.ipynb/Cell 4]

```
model.research_test(strike_level=-1, start=start_date, end=end_date)
```

The Policy Network

In the policy network, `policy.py`, we define a hedging “policy” class on a NN directly, `Policy(nn.Module)`. It takes a three-dimensional state as input, which includes (1) moneyness, defined next, (2) time-to-maturity, and (3) previous hedging position in the underlying stock (e.g. AAPL). And it outputs two parameters, the hedging position for the next step, and the respective uncertainty denoted as *mu* and *sigma* of a normal probability distribution. Note, for code readability, we shall avoid using Greek letters

in the text, except for detailed mathematical equations. Readers should not confuse *mu* and *sigma* of hedging positions with the mean and volatility of asset returns.

Moneyness is defined as the maximum between zero and the difference between the strike price and underlying stock price. For a call option, the option is in the money, that is, moneyness is positive, if the stock price is above the strike price. For a put option, the option is in the money, or moneyness is positive, if the strike price is above the stock price.

A careful reader might have noticed that option price itself is not a feature in the state vector. In the first stage of training, we will simulate option price as a no-arbitrage price based on BSM with GBM of the underlying stocks as discussed previously. Therefore, the option price is totally dependent on the other state variables.

We also did not include other possible features, such as market structure, (e.g., overall trading volume, bid-ask spread, macro-economic backdrops). Readers are encouraged to enrich the state vector with additional features. However, for simulation, you will need to identify and model the data generation process that simulates these additional features.

Let's take a look at the code:

[policy.py]

```
from torch import nn
from torch import optim
from torch import distribution

class Policy(nn.Module):
    def __init__(self, device):
        super(Policy, self).__init__()

        # Define the network layers.
        self._fcin = nn.Linear(3, 256)
        self._fc1 = nn.Linear(256, 256)
        self._fcout = nn.Linear(256, 1*2)

        # Define the optimizer.
        self.optimizer = optim.Adam(self.parameters(), lr=3e-4)

        # Move the network's parameters and buffers to the correct
        # device (CPU or GPU).
        self.to(T.device(device))

    def forward(self, state):
        x = F.relu(self._fcin(state))
        x = F.relu(self._fc1(x))
        x = self._fcout(x)
        lmu, lsig = x.split(1, dim=-1)
        return F.sigmoid(lmu), F.sigmoid(lsig)+1e-12

    def sample(self, state):
        mu, sig = self.forward(state)
        d = D.normal.Normal(mu, sig)
```

```
sample = d.rsample()
return sample, d.log_prob(sample).sum(axis=-1)
```

Here, the policy NN is constructed with three layers. We define the input layer `self.fcin = nn.Linear(3, 256)` to have three input features corresponding to the three state dimensions discussed previously with 256 nodes in this hidden layer. Therefore, there are 256 interim output features from the input layer that will be fed into the nodes of the second hidden layer [Code: `self.fc1 = nn.Linear(256, 256)`]. The second hidden layer then has 256 interim outputs that will be passed to the output layer [Code: `self.fcout = nn.Linear(256, 1*2)`]. And the final output layer consists of the hedging position, **mu** and the uncertainty of the decision **sigma** of the next period as discussed earlier.

The hedging position, **mu**, along with an update to the moneyness and time-to-maturity for the next time period, will be passed **forward** to the next time period to determine the hedging policy **mu**, and **sigma** for the period after next. The ReLU function is used for activation between layers.

Readers are at the liberty of designing their own NN architecture and choosing the appropriate activation function. The author here makes no claim that a three-layer NN is adequate for the complex function of hedging. It is however quite an efficient NN for training a model on simulated data as described next.

Out of the three state dimensions, time-to-maturity is not a random variable, as it will simply be decremented by the length between periods, in this case 1 day, each time we step forward. However, moneyness is a stochastic variable that needs to be calculated based on the dynamics of underlying stock. To capture the dynamics of the underlying stock, we can (a) simulate stock price processes, and (b) use historical stock prices. Option (a) offers the advantage of potentially generating an unlimited amount of data for training purposes. In the follow section, we set up a data generation process using BSM, which assumes stocks follow GBM with fixed drifts, (i.e., average returns), and fixed volatilities of returns. There are more sophisticated models such as Heston (1993) described earlier, where the stationarity of stock return distribution is relaxed by introducing a stochastic process for the volatility itself. For reasons already stated, we shall stick with GBM and hence, BSM, in this example.

Model Functions

The entire model is encapsulated in the `model.py` class, `AIDeltaHedgeModel`. We will not print and describe every definition contained in the entire class, nor is it necessary. We will review the following definitions:

- `_init_()`: class initialization.
- `train_base_model()`: the delta-mimicking training stage with simulated data and a loss function defined as mse between delta and the policy action.

- `train_asset_model()` : the refinement training stage with historical market data and a penalty function based on pnl of the hedged portfolio.
- `research_test()` : a simple test of hedging performance for an option on a single stock for illustration.

[model.py/AIDeltaHedgeModel/_init_0]

```
def __init__(
    self, algorithm, min_contract_duration=timedelta(30),
    max_contract_duration=timedelta(120),
    min_holding_period=timedelta(14), size=(10_000, 1),
    commission=0.01):
    self._algorithm = algorithm
    self._min_contract_duration = min_contract_duration
    self._max_contract_duration = max_contract_duration
    self._min_holding_period = min_holding_period
    self._size = size
    self._commission = commission
    self._pos = 0
    self._policy = None
    self.enable_automatic_indicator_warm_up = True
    # Set the random seeds to enable reproducibility.
    seed_value = 1
    os.environ['PYTHONHASHSEED'] = str(seed_value)
    random.seed(seed_value)
    np.random.seed(seed_value)
    T.manual_seed(seed_value)
    # Set the default tensor type.
    T.set_default_dtype(T.float32)
    # Set the device type (CPU/GPU).
    self._device = 'cuda:0' if T.cuda.is_available() else 'cpu'
```

This initializes the `AIDeltaHedgeModel` object and specifies the maximum/minimum contract duration (120/30 days), minimum holding period (14 days), commission (0.01) used to simulate data, and filter historical data. And it fixes the random seed for reproducibility.

We would like to train our model on a GPU available on [QuantConnect.com](https://www.quantconnect.com) for premium accounts. The last line of code checks to see if a GPU, `torch.cuda` is available. If so, we set the device to `'cuda:0.'` Otherwise, we default to “cpu.”

[model.py/AIDeltaHedgeModel/train_base_model0]

```
def train_base_model(self, plot=True, epochs=1000):
    # Create the policy.
    self._policy = Policy(self._device)
    # Define the loss function.
    mse = nn.MSELoss()

    # Set the volatility and risk-free interest rate arguments.
    # The base model is trained with generated data, which doesn't
    # have a start and end date, so let's just use 2013-2018.
    vol, rf = self._get_vol_and_rf(
        Symbol.create("SPY", SecurityType.EQUITY, Market.USA),
        self._earliest_options_date, datetime(2018, 1, 1))
```

```

)

# Train the model to replicate the underhedged Black-Scholes'
# delta.
in_sample_loss_values = []
oos_loss_values = []
for E in range(epochs):
    # Get the data we need to run the network.
    ttm, moneyness, delta, position = self._generate_data(vol, rf)
    states, y, test_states, test_y = self._forge_batch(
        moneyness, ttm, delta, position
    )
    # Get the in-sample action and loss.
    action, _ = self._policy.sample(states)
    loss = mse(action, y)
    inloss = loss.item()
    # Update the network parameters.
    self._policy.optimizer.zero_grad()
    loss.backward()
    self._policy.optimizer.step()
    # Get the out-of-sample action and loss.
    action, _ = self._policy.sample(test_states)
    outloss = mse(action, test_y).item()
    # Record the loss value of this epoch.
    in_sample_loss_values.append(inloss)
    oos_loss_values.append(outloss)

# Plot the training loss values of each epoch.
if plot:
    x = list(range(1, epochs+1))
    go.Figure(
        [
            go.Scatter(x=x, y=in_sample_loss_values, name='In-sample'),
            go.Scatter(x=x, y=oos_loss_values, name='Out-of-sample')
        ],
        dict(
            title='Training Loss of the Base Model',
            xaxis_title='Epoch', yaxis_title='Loss', showlegend=True
        )
    ).show()

# Save the base model to the Object Store.
joblib.dump(
    self._policy,
    self._algorithm.object_store.get_file_path(self._base_model_key)
)

```

As we discussed, the loss function at this stage is the mean square error (MSE) between BS delta, y , and the policy “action,” as in this line:

```
loss = mse(action, y)
```

where `mse` is assigned `nn.MSELoss()`, which is a method in PyTorch. In each training epoch, we have the following four stages: (1) simulating data for training, (2) splitting data into training and testing sets, (3) training model with the train dataset and validating with test dataset, and (4) calculating respective loss function value. In the `generate_data()` function, we obtain the three state variables (time-to-maturity, moneyness,

and delta), which is then passed into the `forge_batch()` function to split the simulated data into train/test datasets.

The plotting function outputs the convergence graph of the loss function at this stage, which we will discuss in the “Results” section.

Generating Training Data Using Black-Scholes

We define the BS function here. These are standard BSM equations with GBM. The `norm.cdf()` function takes the risk-neutral equivalent of moneyness `d1` and out-of-moneyness `d2` to compute the net risk-neutral expected value of the call option discounted to the present with risk-free rate. The readers are encouraged to look up the derivation of BS formula and BSM model available in numerous textbooks and resources, including an excellent entry on [Wikipedia.org](https://en.wikipedia.org/wiki/Black-Scholes_model).

[model.py/AIDeltaHedgeModel/]

```
def black_scholes_call(S,K,sig,r,t):
    d1 = np.log(S/K) + (r+sig**2/2)*t
    d2 = d1 - sig * np.sqrt(t)
    return S * norm.cdf(d1) - K * np.exp(-r * t) * norm.cdf(d2)

def black_scholes_delta(m,sig,r,t):
    d1 = np.log(m+1.) + (r+sig**2/2)*t
    return norm.cdf(d1)
```

Note, the parameters for the underlying stock distribution and risk-free rate are estimated using historical prices of actual stocks, that is, AAPL, and the primary credit rate from the Federal Open Market Committee as the risk-free rate. We will omit the description of the code that estimates the parameters here as they are straightforward. The code is contained within “model.py.” Price history of the stocks and risk-free rate are retrieved from market data on [QuantConnect.com](https://www.quantconnect.com), using predefined data query functions available on the platform. Without loss of generality, we will train the policy network by considering a fixed volatility level of the individual stock (i.e., AAPL in our example). It is entirely likely that the parameters of the best hedging policy may depend on the stock’s volatility. Readers are encouraged to train their models on multiple stock volatility assumptions.

The functions for generating training data and splitting data into train and test subsets are defined next. Note the features that define the state vector is forged into a tensor using PyTorch’s method `tensor()`.

[model.py/AIDeltaHedgeModel/]

```
def generate_data(size):
    ttm = np.random.uniform(size=size) * 31 / 252
    moneyness = np.random.uniform(size=size) * 2 - 1
    delta = black_scholes_delta(moneyness,vol,rf,ttm)
    position = np.random.uniform(size=size)
    return ttm, moneyness, delta, position
```

```
def forge_batch(moneyness, ttm, delta, position, ratio=0.75):
    cut = int(moneyness.size * ratio)
    tmp = delta * 0.9 + position * 0.1
    states = T.tensor(np.concatenate((moneyness.flatten()[:cut,np.newaxis],np.sqrt(ttm.flatten()))[
    y = T.tensor(tmp.flatten()[:cut,np.newaxis]).float().to(DEVICE)

    test_states = T.tensor(np.concatenate((moneyness.flatten()[cut:,np.newaxis], np.sqrt(ttm.flatten()
    test_y = T.tensor(tmp.flatten()[cut:,np.newaxis]).float().to(DEVICE)

    return states, y, test_states, test_y
```

Target and Loss Function with Simulated Data

To recap, in this stage of the learning algorithm, we allow the policy network to learn or “mimic” the behavior of “under-hedged” Black-Scholes delta. The term “under-hedged” refers to the fact that the simulated and real hedging strategies are discrete and not continuous. Since delta changes continuously as the underlying stock’s volatility and price change, any discrete delta-hedge is insufficient to match the PnL of the option position.

Here, the simulation assumes a BSM process, and therefore, the BS delta is used as a target for learning. In other words, in this step, the policy network will learn to estimate delta, without knowing the underlying mathematics. One can also train the policy network by targeting the PnL of the option position, in which case the policy network will learn the BS prices. In practice, delta is a first derivative of stock price, and it is bounded between 0 and 1 for a call or put option. The policy network will converge more quickly and more robustly with a bounded target like delta.

To generalize the method here with other (derivatives, underlying) pair dynamics that do not have a closed form delta, one can compute delta numerically using the finite-difference method from simulated prices. It is important to note that the policy learned will not be identical to deltas at present nor next period, as the policy hedge ratio for the next period is determined based on the current state (see earlier discussion). Given the stochastic nature of underlying stock, one can only assess the probability distribution of the next period delta, and therefore, there will be finite loss between policy hedge ratio and the BS delta. The example code minimizes MSE, between the policy “action” and target delta, “y.”

To avoid confusion, it may be helpful to point out that the purpose of this stage of training is not simply to learn delta from simulated data. It is to “prime” the policy network by mimicking the behavior of delta hedging. Subsequent to learning “delta,” the policy network will then learn from historical market data. At this second stage of the training, we will specify a “penalty” function that is based on the variance of the PnL of the hedged portfolio to fine tune, or “reinforce” the policy network.

Fine-tuning with Market Data

In this section, we will fine-tune or reinforce the policy network, with recent historical market data. The purpose of this training stage is to inject

a source of “realism.” As the previous stage trains the policy network to mimic the behavior of delta hedging, the policy network has only learned the “textbook” rules of hedging. In this stage, we train the policy network on actual market prices, which are presumed to incorporate recent market conditions, such as transaction costs, liquidity, and market trends. In order to fully incorporate information latent in market prices, we define a penalty function on the change of the PnL of the hedged portfolio. The objective of the refinement learning is to minimize the changes in PnL, defined as follows:

$$\Delta_{t+1} = a_{t+1} S_{t+1} - S_t - c * a_{t+1} - a_t - C_{t+1} - C_t$$

where Δ_{t+1} is the “change” in the total value of the hedged portfolio between time t and $t + 1$; a_{t+1} , which is the hedging action, or quantity of the underlying stocks to hold, determined at time t and held until $t + 1$; S_t , which is the price of underlying stock at time t ; c is the commission rate of trading the underlying stock, and C_t is the “invoice” price of the call option. Note, the call’s invoice price is adjusted from the quoted price by multiplying the quoted price by 100, as per convention of listed options. The loss function is computed by applying a ReLU function to the negative of the “change” in PnL, that is, Δ_{t+1} as a percentage of “wealth,” that is, total value of the hedged portfolio, written in code:

```
penalty = T.relu( - change / wealth)
wealth += change.item()
```

The use of ReLU functional form for the penalty function is an efficient method as ReLU is used as an activation function in the refinement stage. This makes the activation more direct and loss function more likely to converge, as there is no need to compute a separate gradient of descent on the loss function. In addition, the ReLU function penalizes negative changes in wealth but not positive changes. This has the benefit of encouraging learning of potential “alpha” opportunities. Although the objective of hedging is not to generate alpha, it never hurts to pick up the dollar bill lying on the sidewalk when we see one.

Pick Your Poison: Price or Delta?

More seriously, we will pause here to discuss an important design decision in reinforcement learning for hedging options. The central challenge of any training program is an unstable and unbounded target or loss function that does not converge. Option and stock prices are unbounded, and the option price is a non-linear function of the underlying stock price. Even if the underlying stock returns are GBM, which implies that the return is normally distributed, the option prices and returns are not normally distributed, and has extreme skew and kurtosis. Therefore, a PnL-based loss function will likely require large amounts of high-quality data, a (very) large policy network, and many epochs of training to converge, if at all. In order to reduce the variance of loss functions, several choices are available.

First, regularization, a method of “clipping” the loss function in the context of Q-learning, has been tested in academic research. It has been

shown to stabilize loss functions during training and speed up convergence (e.g., Du et al., 2020). This is the chosen method on the research platform at the author's firm, Adaptive Investment Solutions, LLC. This is only feasible if the researcher has access to a large amount of high-quality historical market data, and high-end machine-learning processors and GPUs.

The second choice is what we are presenting in this book. Here as we have demonstrated, we start with a policy network primed with “delta hedging” using simulated data. That is, the policy network has already gone to “school” to learn the delta from BSM. We then fine-tune the policy network with recent market data. Effectively, this is a two-step regularization procedure: the policy network is primed with a prior that mimics delta hedging in the first step and will then learn to adjust its actions based on recent market data in the second step. One consequence of this design is that the policy network will be more dynamic but also potentially make more mistakes, that is, taking recent history as an indication of the future. However, the dynamic change in policy is small compared with the core policy behavior of discrete delta hedging. We can make the policy network more stable by simulating more data in the first training stage, if we desire. Figure 7.2 illustrates the two stages of consecutive training and the process of updating the policy forward in time.

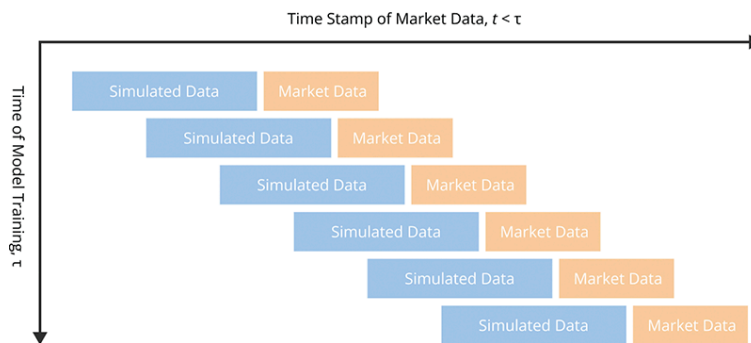


Figure 7.2 Illustrative diagram of the continuous two-stage RL. The RL model can be updated and retrained continuously. At each model update/retraining at time, τ , we will use a short window of market option prices date at $\tau - \Delta t < t < \tau$ for the refinement stage. The simulated dataset can also be updated using a longer historical window of stock prices prior to $\tau - \Delta$ to determining the distribution of stock returns.

Lastly, it is possible to combine the two loss functions into one and train the model on a large amount of historical market data. That is, construct a loss function with a term to minimize the variance between the policy and delta hedging and another term minimizing the variance of the PnL of the hedged portfolio, with a regularization factor to reduce the influence of the PnL term. This method will require hyperparameter tuning. If applied to only historical data, the parameter tuning could result in in-sample overfitting. We did not adopt this method.

Refinement with Real Data

The second stage of training is the refinement of policy network with real market data. The code example for the second stage,

```
train_asset_model()
```

is shown here.

```
[model.py/AIDeltaHedgeModel/]
```

```

def train_asset_model(
    self, ticker, start_date, end_date, epochs=20,
    save=False, in_research_env=False):
    # If there is no base model in the Object Store yet, add one.
    if not self._algorithm.object_store.contains_key(self._base_model_key):
        print("No base model in the Object Store. Let's create one first.")
        self.train_base_model()

    # Adjust the QuantBook date to avoid look-ahead bias.
    if in_research_env:
        self._algorithm.set_start_date(end_date)
    # Add a security initializer so that `self._equity.price`
    # isn't zero in research and so we can trade Option contracts
    # right after we subscribe to them while trading.
    self._algorithm.set_security_initializer(
        BrokerageModelSecurityInitializer(
            self._algorithm.brokerage_model,
            FuncSecuritySeeder(self._algorithm.get_last_known_prices)
        )
    )
    # Subscribe to the underlying Equity.
    self._equity = self._algorithm.add_equity(
        ticker, data_normalization_mode=DataNormalizationMode.RAW
    )
    # Create a member on the Equity object to track the current
    # Option contract.
    self._equity.option_contract = None

    self._asset_model_key = "ai-hedging-" + ticker
    epoch_penalties = self.refit(save, epochs)
    # Plot the training penalties.
    if in_research_env:
        go.Figure(
            go.Scatter(
                x=list(range(1, epochs+1)), y=epoch_penalties,
                name='Total Penalty'
            ),
            dict(
                title='Training Penalties of the Refined Model<br><sub>'
                    + f'Refined the base model for {ticker}</sub>',
                xaxis_title='Epoch', yaxis_title='Total Pentality',
                showlegend=True
            )
        ).show()

    if not in_research_env:
        return self._equity.symbol

```

The code is self-explanatory. The function `refit()` is used to compute the penalty function at each epoch as described earlier:

[model.py/AIDeltaHedgeModel/]

```

def refit(self, save=False, epochs=20, lookback=timedelta(2*365)):
    # Load the base model from the Object Store.
    self._policy = joblib.load(
        self._algorithm.object_store.get_file_path(self._base_model_key)
    )
    contract_list_date = self._algorithm.time - lookback

```

```

# Train the base model to be specific to this asset.
contract_symbols, selected_strikes, data = self._get_data(
    None, contract_list_date, self._algorithm.time
)
# Reset the optimizer and run the epochs.
self._policy.optimizer = optim.AdamW(self._policy.parameters(), lr=1e-5)
epoch_penalties = []
for E in range(epochs):
    total_pen = 0
    for k in selected_strikes:
        # Get the closest expiry.
        next_expiry = min(
            [
                symbol.id.date for symbol in contract_symbols
                if symbol.id.strike_price == k
            ]
        )
        # Get the Option sequence data.
        path = self._get_option_seq(
            data, k, next_expiry, contract_list_date,
            self._algorithm.time
        )
        s = path['close_underlying']
        c = path['close_option']
        if c.empty():
            continue
        moneyness = s / k - 1
        ttm = np.array(
            [path.shape[0] - i for i in range(path.shape[0])]
        ) / 252
        position = T.zeros([1], device=self._device, dtype=T.float32)
        wealth = c.iloc[0]

        self._policy.optimizer.zero_grad()

        for t in range(path.shape[0]-1):
            state = T.cat(
                (
                    T.tensor(
                        [moneyness.iloc[t], ttm[t]],
                        device=self._device, dtype=T.float32
                    ),
                    position
                )
            ).unsqueeze(0)
            new_position, _ = self._policy.sample(state)
            change = (
                new_position * (s.iloc[t+1] - s.iloc[t])
                - self._commission * (new_position - position)
                - (c.iloc[t+1] - c.iloc[t])
            )
            penalty = T.relu(- change / wealth)
            wealth += change.item()
            (penalty).backward()
            total_pen += penalty.item()

        self._policy.optimizer.step()
        epoch_penalties.append(total_pen)

# Save the model to the Object store.
if save or self._algorithm.live_mode:

```

```

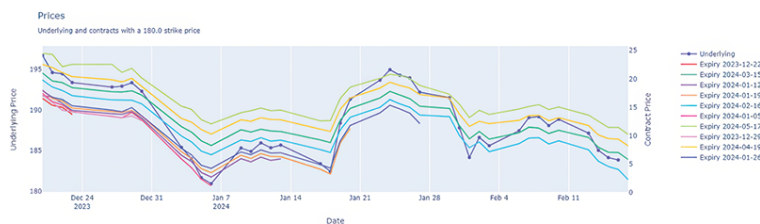
        joblib.dump(
            self._policy,
            self._algorithm.object_store.get_file_path(self._asset_model_key)
        )
    return epoch_penalties

```

Results

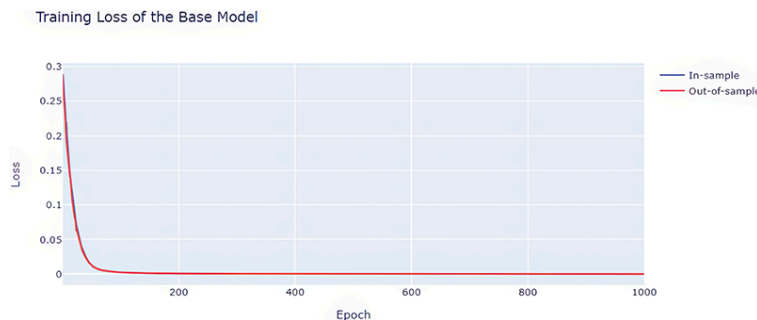
The code for forward testing is straightforward and is contained within `model.py`, which is published in the book's GitHub repository along with other project files used in this example. Here we shall summarize some results, illustrated by the graphs plotted in the training and testing phases.

This demonstration code uses AAPL stock and AAPL call option prices to train the policy network. [Figure 7.3](#) shows the daily closing price of AAPL and AAPL call options from December 2023 and February 2024, the period used for the refinement part of the training. The training is based on the front call options (live options with the closest expiry). As front call options expire, the price series switches to the contract with the next closest expiry. As the graph demonstrates, there is a price discontinuity at each roll. This is not a problem for the RL, as the state variables are updated with the next contract's expiration. Implicitly, calendar roll is in the training data, and the RL may incorporate price dynamics specific to calendar rolls when tuning the policy network.



[Figure 7.3](#) Daily closing price of AAPL (Apple, Inc.) shares and AAPL Call options with 180 Strike from December 2023 to February 2024.

The convergence of the loss function in the first stage of delta-mimicking training on simulated data is rapid, as shown in [Figure 7.4](#).



[Figure 7.4](#) Convergence of loss function of the base model, that is, delta-mimicking stage of training. Recall the loss function is the MSE between policy action and delta of the options computed with simulated data. The graph shows rapid convergence both in-sample and out-of-sample. Note, it is not surprising that the convergence of the in-sample and out-of-sample loss functions appear identical. This is because, at this stage, the model is trained on simulated data that are identically distributed between in-sample and out-of-sample.

In the second refinement stage of training with historical market data, the penalty function also converges in a robust manner as [Figure 7.5](#)

shows.

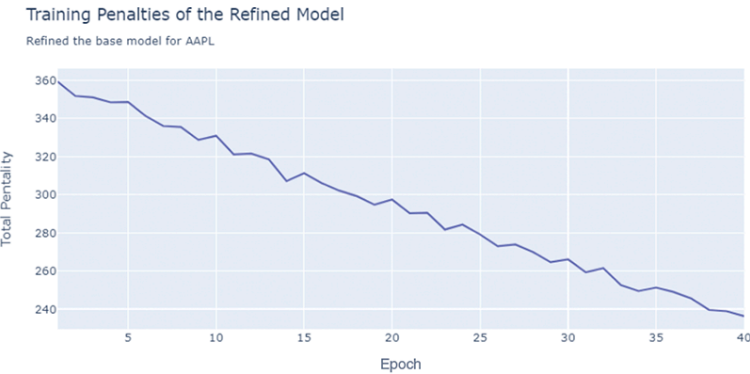


Figure 7.5 Convergence of the PnL-based penalty function (loss function) during the refinement stage. This shows a rapid convergence of penalty function.

Figure 7.6 compares the hedging decision (or action) from the trained policy network to BS delta. The plot shows that the hedging decisions in this case are correlated with BS delta, but the AI policy deployed a lower hedge ratio than the theoretical delta. This makes sense, as in discrete hedging, there is a large variance of potential outcome in the future prices of stocks and call options, as well as delta values. The uncertainty pushes the policy network to take a smaller position in the underlying, since it is less costly to catch up than overshoot the ideal delta, when trading is costly. Also, AAPL price has been in decline earlier in this period. Since delta of a call option declines when underlying stock price decline, the effective forward delta will have been lower than theoretical spot delta.

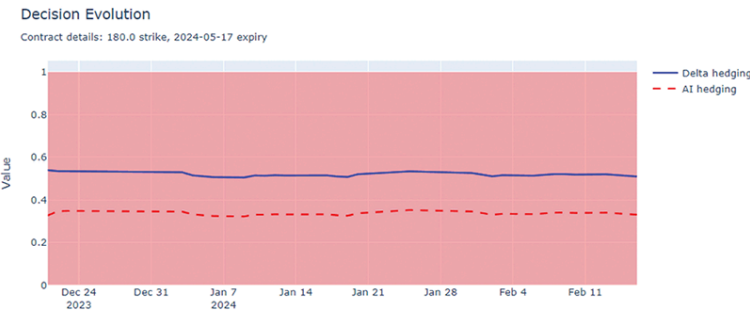


Figure 7.6 Comparison of the decision evolution of the full model compared to the evolution of delta of an AAPL call with a 180 strike and a 2024-05-17 expiry. In this case, AI hedging policy appears to under-hedge compared delta. This is likely as AAPL prices have been in decline earlier in this period. Therefore, the effective forward delta would have been lower than theoretical spot delta.

The following two charts, **Figures 7.7** and **7.8**, show comparisons of the wealth of the hedging portfolio (i.e., the stock portfolio) and the wealth of the hedged portfolio (i.e., call option + stock). These illustrate that the AI hedging policy outperforms all alternative hedging policies. It also has lower variance than numerical hedging, and similar variance to Delta hedging strategies (Hold and Discrete Delta hedging).

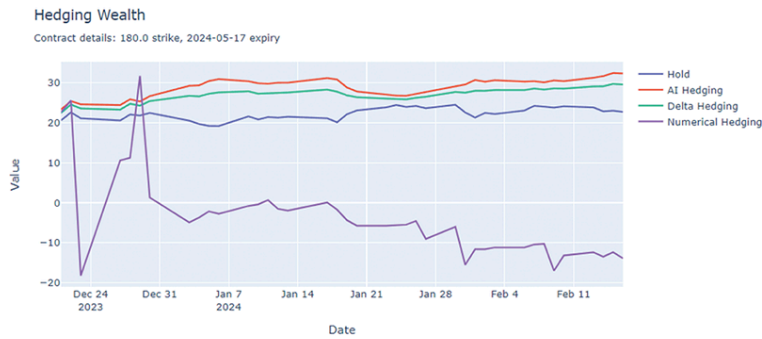


Figure 7.7 Cumulative changes of the total wealth of the hedging portfolios of static hedging, that is, “Hold” the original hedging position, the AI hedging portfolio, the delta hedging portfolio, and a numerical hedging portfolio (i.e., hedged with delta calculated using finite difference method with market data).

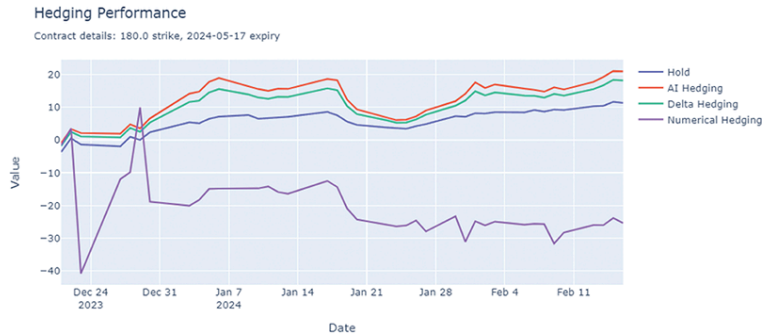


Figure 7.8 Cumulative changes of the net wealth of the hedged portfolio (i.e., the option + hedging position) of static hedging (“Hold”), the AI hedging portfolio, the delta hedging portfolio, and a numerical hedging portfolio (i.e., hedged with delta calculated using finite difference method with market data).

Conclusion

AI hedging is still in its infancy but has already demonstrated huge potential. This chapter demonstrates that with some care, simple reinforcement learning models can be implemented for practical use. The two-stage process of first priming the policy network with a textbook delta hedging policy on simulated data, followed by refining the policy with a PnL-based penalty function using actual market data, works quite well for small-scale learning models. The methodology is agnostic to the assumptions of the underlying stochastic dynamics of the hedged and hedging assets. Therefore, readers can test various stochastic models for the underlying assets.

A bonus of a robust hedging policy is that, through finding the best optimal hedging policy, one may shed some light on the true underlying security price processes as well, and in turn help with refining the pricing model for the hedged and hedging assets. Indeed, in some advanced RL models of AI hedging, a “pricing” kernel is a natural output of the hedging policy and can be used to predict market prices of options.

Another encouraging result is, through machine learning, we are able to incorporate realistic market conditions, by simply “refining” the learning process on market data. The policy network adjusts to market conditions consistent with the sample data. This is because the hedging policy is agnostic to any structural model of option prices. Therefore, any known or unknown non-fundamental processes or innovations in option prices are admissible, and potentially “learnable.” Of course, market conditions evolve over time. It is important to update the model forward. This

rolling forward strategy is an age-old learning and fine-tuning strategy of quant finance. We simply updated it for this new generation of AI agents.